LECTURE 7

Synthesis with constraints

The techniques described in the previous lecture used symbolic representations of the program space, but they still involved a fair amount of enumeration. We now focus on a class of techniques that are "more symbolic", and have more flexibility in capturing complex program spaces, although at the expense of a significant computational cost.

For this lecture and the next, we will be using the Sketch synthesis system as a canonical example [1], although there are other systems such as Rosette [2], SyGuS solvers such as CVC5 [3], and Brahma [4] which are based on similar principles. The similarities and differences between these different systems will be elaborated at the end of the unit.

7.1 Constraint-based synthesis at a glance

The high-level idea in constraint-based synthesis is to represent the program space as a parametric program $P[c]$, so that different values of the parameters correspond to different programs in the space. The idea is to translate requirements on the behavior of the program $P[c]$ into constraints on the parameters c, so that any value of c that satisfies the constraints $\varphi(c)$ is guaranteed to lead to a program $P[c]$ satisfying all the requirements.

In order for this approach to work, we need three ingredients. First, we need a mechanism for creating parametric programs from a high-level definition of the program space. Second, we need a mechanism for constructing constraint systems from these parametric programs and their requirements, and finally, we need efficient mechanisms for solving the resulting constraint systems. We start by addressing the first point.

7.2 From program spaces to parametric programs.

There are two major approaches for defining the parametric programs that are the starting point of constraint-based synthesis. The first approach is to provide the user with a high-level notation for describing a program space, and then have a compiler that converts this definition into a parametric program. This is the approach taken by Brahma or by the SyGuS solvers. In the case of Brahma, the user simply provides a bag of components, and the system automatically produces a parametric program where different choices of parameters correspond to different ways of connecting the components together. In

the case of the SyGuS solvers, the user provides a context-free grammar for a space of expressions, and the solver generates a parametric program from this grammar.

The alternative approach, implemented in Sketch, is to provide the user with a rich and expressive language for directly writing parametric programs. This expressiveness provides the programmer with significant control over the program space and its encoding as a parametric program. That control allows an expert user to carefully engineer a program space to maximize the efficiency of the synthesis process, but it also introduces an extra level of complexity for less sophisticated users who must deal with the added complexity of defining their program space as a parametric program. Sketch tries to alleviate this burden by providing powerful abstraction facilities that allow potentially complex definitions of program spaces to be encapsulated and reused across many different programs.

7.3 Sketch: a language for parametric programs.

The most authoritative source for the sketch language is the sketch manual. In this section, we provide a brief overview of the key principles behind the language.

At a high-level, sketch is a simple imperative language with support for many of the features we have come to expect from modern languages including heap allocated structures, high-order functions and polymorphism (known as generics in Java). There are three features, however, that distinguish Sketch from other languages: Unknown constants, harnesses and generator functions.

Unknown constants. An unknown constant in Sketch is expressed as ??. The type of this constant is inferred from context; it can be an integer, a boolean, a character or a fixed size array of either. At synthesis time, sketch replaces each unknown constant with a fixed constant so that all the requirements are satisfied. For example, the simplest sketch program that illustrates the main ideas in the language is shown below.

```
int doublevalue(int in){
    int t = in * ??;
    assert t == in + in;
    return t;
}
```

In the program, the unknown constant must be replaced with an integer constant. The assertion imposes the requirement that `t==in+in`, which clearly forces the unknown constant to resolve to the number 2. The assertion, however, is only valid in the context of a *test harness*.

Test harnesses. A test harness is simply a function that when invoked must not trigger any assertion violations. For example, in order to force the

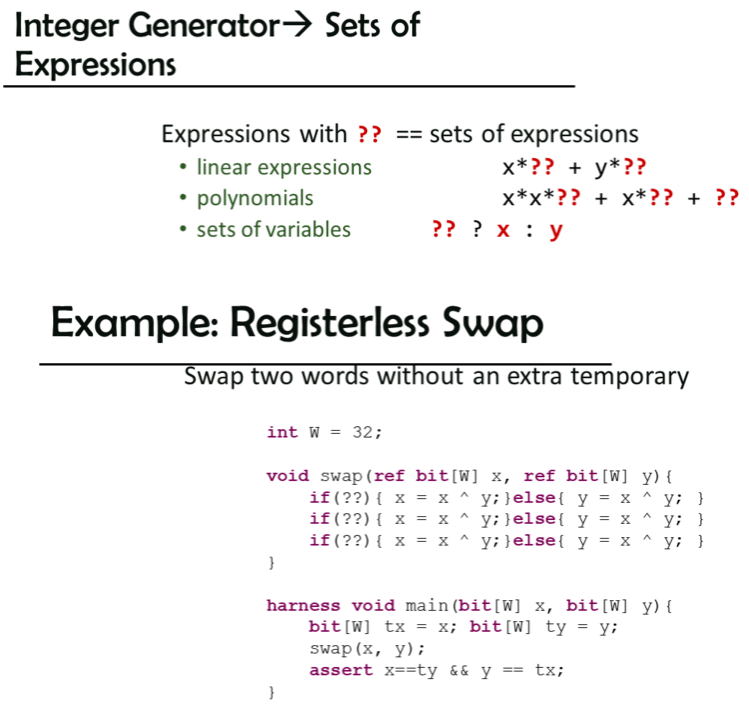
`doublevalue` function above to synthesize to the correct function, we can use the following test harness.

```
harness void test1(){
        doublevalue(5);
        doublevalue(7);
        doublevalue(3);
}
```

We could have also excluded the `assert` inside the `doublevalue` function itself and instead placed the assertion in the test harness.

```
harness void test1(){
        assert doublevalue(5) == 10;
        assert doublevalue(7) == 14;
        assert doublevalue(3) == 6;
}
```

Since we are focusing on the inductive synthesis case, we will focus on the case where the test harness does not take any inputs, and instead just invokes the desired functions using fixed values. Later in the course we will explore more general test harnesses that can impose constraints that must hold for all inputs.



Generator function. At this point, we already have a language expressive

enough to discover some interesting aspects of a program. For example, if a program involves an affine expression, say over a variable x, but we do not want to have to think about the constants involved, we can just express it as x*??+??. Or, for example, if at some point we are not sure whether we should use variable x or variable y, we can use ?? ? x : y, using the ?: ternary operator like the one available in C. In order to support the description of more general program spaces, we need some additional machinery, which we borrow from the generative programming literature. In particular, Sketch uses the notion of a *generator*, which looks like a function, but with the property that it will get fully inlined and partially evaluated into its calling context.

As a simple example directly from the Sketch manual, consider the problem of specifying the set of linear functions of two parameters x and y. That space of functions can be described with the following simple generator function:

```
generator int legen(int i, int j){
   return ??*i + ??*j + ??;
}
```

The generator function can be used anywhere in the code in the same way a function would, but the semantics of generators are different from functions. In particular, every call to the generator will be replaced by a concrete piece of code in the space of code fragments defined by the generator. Different calls to the generator function can produce different code fragments. For example, consider the following use of the generator.

```
harness void main(int x, int y){

  assert legen(x, y) == 2*x + 3;
  assert legen(x,y) == 3*x + 2*y;

}
```

Calling the solver on the above code produces the following output

```
void _main (int x, int y){
  assert ((((2 * x) + (0 * y)) + 3) == ((2 * x) + 3));
  assert (((3 * x) + (2 * y)) == ((3 * x) + (2 * y)));
}
```

Note that each invocation of the generator function was replaced by a concrete code fragment in the space of code fragments defined by the generator.

Sample Generator

```
/**
 * Generate the set of all bit-vector expressions
 * involving +, &, xor and bitwise negation (~).
 * the bnd param limits the size of the generated expression.
 */

generator bit[W] gen(bit[W] x, int bnd){
    assert bnd > 0;
    if(??) return x;
    if(??) return ??;
    if(??) return ~gen(x, bnd-1);
    if(??){
        return {| gen(x, bnd-1) (+ | & | ^) gen(x, bnd-1) |};
    }
}
```

High order generators

```
/*
 * Generate code from f n times
 */
generator void rep(int n, fun f){
    if(n>0){
        f();
        rep(n-1, f);
    }
}
```

Up to this point, though, the generator may seem like a typesafe macro, little more than syntactic sugar. What gives generators their real power is the ability to be recursive. For example, the generator in the figure, describes a grammar of expressions, which can either be a variable `x`, an unknown bit-vector constant, or the bitwise combination or bitwise negation of recursively generated expressions. Each recursive invocation of the generator can have its own distinct values for the unknown constants. This idiom of using generators to define a space of programs as a context free grammar is quite common across many different applications of Sketch.

In addition to being recursive, generators can also be high-order, meaning that they can take other functions or even other generators as parameters. An example of this is the `rep` generator also shown in the figure. This generator takes as a parameter a function or a generator `f`, and applies it n times.

```
generator void rep(int n, fun f){
    if(n>0){
        f();
        rep(n-1, f);
```

```
    }
}
```

This very simple generator actually implements a very important computational pattern, one where a particular kind of operation needs to be performed multiple times, but each time may actually correspond to a distinct operation.

For example, consider the code below:

```
bit[32] reverseSketch(bit[32] in) {
    bit[32]  t = in;
    int s = 1;
    generator void tmp(){
        bit[32] m = ??;
        t = ((t << s) & m )| ((t >> s) & (~m));
        s = s*??;
    }
    rep(??, tmp);
    return t;
}
```

The goal of the sketch above is to reverse the bits in a 32-bit word through a combination of shifts and masks. The generator `tmp` reflects the basic computational pattern for each step, where the word is shifted left and right by some amount, and a mask determines which bits to keep from the left shift and which from the right shift. After that, the shift amount is multiplied by a constant. We know the computation involves some number of such operations, but not how many. The generator `rep` is ideally suited for that purpose. Note that the first parameter `n`, which defines the depth of the recursion, does not have to be a constant; the number of iterations is part of what the synthesizer needs to discover.

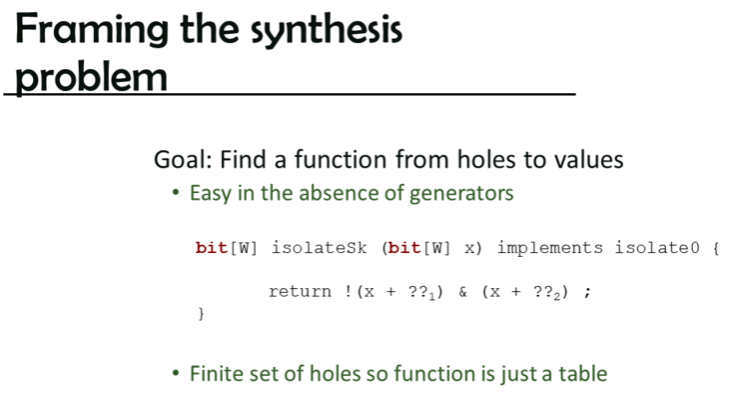
The result of solving this sketch against a suitable harness would look something like this:

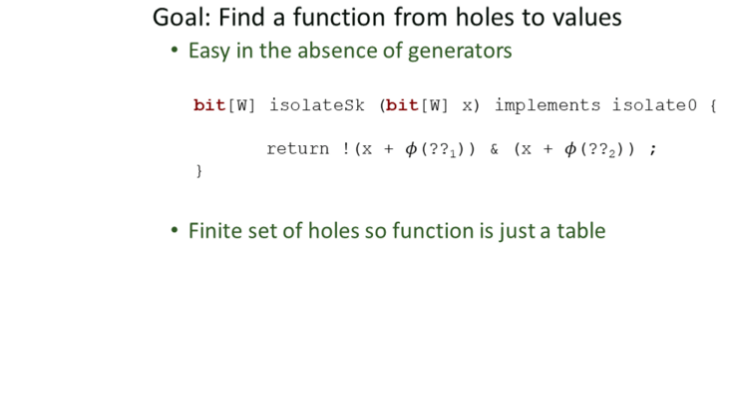
```
void reverseSketch (bit[32] in, ref bit[32] _out)  implements reverse/*
    reverse.sk:7*/
{
  bit[32] __sa0 =
    {0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1};
  _out = ((in << 1) & __sa0) | ((in >> 1) & (~(__sa0)));
  bit[32] __sa0_0 =
    {0,0,1,1,0,0,1,1,0,0,1,1,0,0,1,1,0,0,1,1,0,0,1,1,0,0,1,1,0,0,1,1};
  _out = ((_out << 2) & __sa0_0) | ((_out >> 2) & (~(__sa0_0)));
  bit[32] __sa0_1 =
    {0,0,0,0,1,1,1,1,0,0,0,0,1,1,1,1,0,0,0,0,1,1,1,1,0,0,0,0,1,1,1,1};
  _out = ((_out << 4) & __sa0_1) | ((_out >> 4) & (~(__sa0_1)));
  bit[32] __sa0_2 =
    {0,0,0,0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,1,1,1,1,1,1,1,1};
  _out = ((_out << 8) & __sa0_2) | ((_out >> 8) & (~(__sa0_2)));
```

```
  bit[32] __sa0_3 =
    {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1};
  _out = ((_out << 16) & __sa0_3) | ((_out >> 16) & (~(__sa0_3)));
  return;
}
```

An important thing to note in the generated code is that much of the control structure in the generator has completely disappeared. All the branches and all the recursive calls have been partially evaluated away, so what is left is just the code the user actually wants, albeit with somewhat ugly variable names.

7.4 Formalizing generator functions.





Framing the synthesis problem

Generators need something more

- The value of the holes depends on the context

```
generator bit[W] gen(context τ, bit[W] x, int bnd){
    assert bnd > 0;
    if(φ(τ,??1)) return x;
    if(φ(τ,??2)) return φ(τ,??5);
    if(φ(τ,??3)) return ~gen_g1(τ·g1, x, bnd-1);
    if(φ(τ,??4)){
        ...
    }
}

bit[W] isolate0sk (bit[W] x)  implements isolate0 {
     return gen_g0(g0, x, 3);
}
```

Framing the synthesis problem

```
generator bit[W] gen(context τ, bit[W] x, int bnd){
   assert bnd > 0;
   if(φ(τ,??1)) return x;
   if(φ(τ,??2)) return φ(τ,??5);
   if(φ(τ,??3)) return ~gen_g1(τ·g1, x, bnd-1);
   if(φ(τ,??4)){
     return {| gen_g2(τ·g2, x, bnd-1) (+ | & | ^) gen_g3(τ·g3, x, bnd-1) |};
   }
}

bit[W] isolate0sk (bit[W] x)  implements isolate0 {
    return gen_g0(g0, x, 3);
}
```

- Potentially unbounded set of unknowns
- We can bound the depth of recursion
  - That means again ϕ is just a table

$\phi(g_0, ??_k)$
$\phi(g_0g_1, ??_k)$
$\phi(g_0g_2, ??_k)$
$\phi(g_0g_1g_2, ??_k)$
$\phi(g_0g_1g_3, ??_k)$
$\phi(g_0g_1g_2g_1, ??_k)$
...

The first step in understanding how constraints are produced from a program written in Sketch is to state more precisely the semantics of generators. This will be done following the formalization in a 2013 journal paper on Sketch [5]. As mentioned before, Sketch is really just a notation for writing parametric programs. A program in Sketch can be thought of as a parametric function, parameterized by a function ϕ. This function ϕ is really just a table that tells us the value of each of the different constants inside a Sketch.

If it were not for recursive generators, it would be straightforward to simply assign a unique name to each distinct unknown constant in the sketch, and then make ϕ just a mapping from that name to a corresponding value as illustrated in the figure. However, recursive generators introduce a wrinkle into this story because the same syntactic instance of a hole is supposed to have different val-

ues for different instances of the generator. We formalize this by making ϕ a function of a *context* in addition to a hole.

The idea is that when you write a sketch with generators, the compiler internally assigns each callsite for a generator a unique code. When a generator is called, it is assigned a calling context which summarizes where the generator was called. This context is then passed to ϕ as illustrated in the figure. Note that when a generator is called from another generator, the new callsite name is appended to the existing context, but when it is called from a function, there is no prior context to consider, and only the callsite name is used. Also, if a hole is used outside of a generator, just within a normal function, then it will just have the empty context.

The result, as illustrated in the last frame in the figure, is that a sketch with recursive generators can have a potentially unbounded set of values. In practice, Sketch avoids this by bounding the depth of recursion for generators, a bound that is defined by a command-line flag `"-bnd-inline-amnt"`. With a bound on the depth of recursion ϕ becomes once again just a table, mapping a finite set of hole names and a calling contexts to values. Over the next lecture, then, we focus on the process of generating constraints on ϕ, and solving them in order to find values that allow the sketch to satisfy all its assertions.

7.5 From sketch to constraints

Before we jump into the details of how to generate constraints from a Sketch, we need to understand something about how to define the semantics of a simple language. There are many different formalisms for doing this, but a very popular one in the context of imperative language is to define the semantics of an expression as a function from a state to a value. Specifically, the *denotational semantics* for expressions is a function

$$\mathcal{A}[\![\cdot]\!] : Expr \rightarrow \Sigma \rightarrow Val$$

Where Σ is just the set of possible states for a program. In other words, for a given expression, say $x + 5$, $\mathcal{A}[\![x + 5]\!]$ will give us a function that given a state containing values for each variable, will give us back a value, hopefully corresponding to the value of x in the state plus five.

Similarly, the semantics of a statement (also often called command in the literature) are represented as a function that maps the state of a program to a new state. Specifically, the denotational semantics for a statement is a function

$$C[\![\cdot]\!] : cmd \rightarrow \Sigma \rightarrow \Sigma$$

Semantics of a simple language

e:= n | x | e_1 + e_2 | e_1 > e_2
c:= x := e | c_1 ; c_2 | if e then c_1 else c_2 | while e do c

What does an expression mean?

- An expression reads the state and produces a value
- The state is modeled as a map σ from vars to values
- $\mathcal{A}[\![\cdot]\!] : e \to \Sigma \to int$

Ex:

- $\mathcal{A}[\![x]\!] = \lambda\sigma.\sigma(x)$
- $\mathcal{A}[\![n]\!] = \lambda\sigma.n$
- $\mathcal{A}[\![e_1 + e_2]\!] = \lambda\sigma.\mathcal{A}[\![e_1]\!]\sigma + \mathcal{A}[\![e_2]\!]\sigma$
- $\mathcal{A}[\![e_1 > e_2]\!] = \lambda\sigma.if\ \mathcal{A}[\![e_1]\!]\sigma > \mathcal{A}[\![e_2]\!]\sigma\ then\ 1\ else\ 0$

Semantics of a simple language

e:= n | x | e_1 + e_2
c:= x := e | c_1 ; c_2 | if e then c_1 else c_2 | while e do c

What does a command mean?

- A command modifies the state
- $\mathcal{C}[\![\cdot]\!] : c \to \Sigma \to \Sigma$

Ex:

- $\mathcal{C}[\![x := e]\!] = \lambda\sigma.\sigma[x \to (\mathcal{A}[\![e]\!]\sigma)]$
- $\mathcal{C}[\![c_1; c_2]\!] = \lambda\sigma.\mathcal{C}[\![c_2\ ]\!](\mathcal{C}[\![c_1\ ]\!]\sigma)$
- $\mathcal{C}[\![if\ e\ then\ c_1 else\ c_2]\!] =$
  $\lambda\sigma.\lambda x.if\mathcal{A}[\![e]\!]\sigma = 1\ then\ (\mathcal{C}[\![c_1\ ]\!]\sigma)x\ else(\mathcal{C}[\![c_2\ ]\!]\sigma)x$

What about loops?

Semantics of a while loop

- Let $W = \mathcal{C}[\![while\ e\ do\ c]\!]$
- W satisfies the following equation:
  $W = \lambda\sigma.\lambda x.if\mathcal{A}[\![e]\!]then\ \big(W(\mathcal{C}[\![c\ ]\!]\sigma)\big)x\ else\ \sigma x$
- Equation can have many solutions
  - when loop doesn't terminate
- Rich theory for finding least fixed point solution
- We'll settle for a simpler strategy:
  unroll k times and then give up

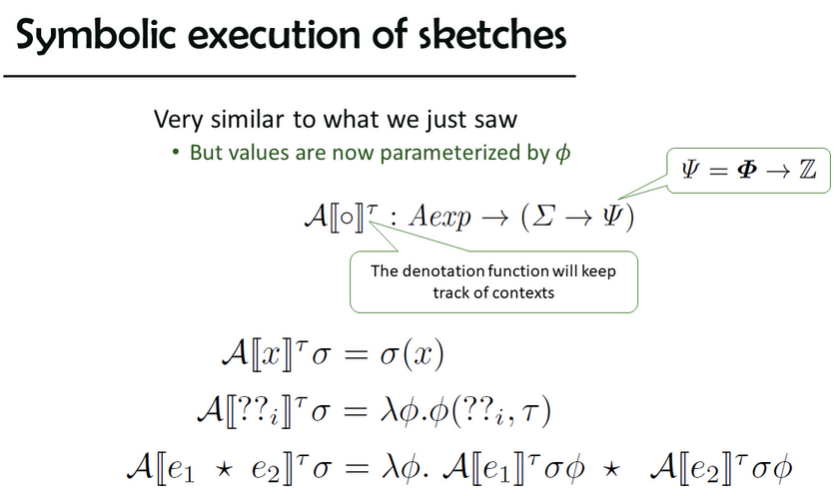
For example, the figure illustrates the semantics for a very simple imperative language. Note that the semantics are described recursively following the

syntactic structure of the language. For example, the semantics of constants are just the value of the constant, and the semantics for variables are defined in terms of the state, which assigns a value to every variable. For simplicity, the language elides the distinction between booleans and integers, just using 1 as a stand-in for true.

For commands, the semantics are also defined recursively. The most interesting rule is the rule for assignment of the form `x:=expr`, which produces a new state that is just like the state before the assignment, but with x now mapped to a new value corresponding to the result of evaluating $expr$ on the initial state. Sequential composition is just what one would expect, the semantics is just the result of chaining together the semantics of the two corresponding statements. The rule for `if` is interesting notationwise. To understand the notation, remember that a state is a mapping from variable names to values. So the if rule produces a new mapping that will return either the values created by the *then* branch, or the values created by the *else* branch depending on whether the condition evaluated to true or not.

Loops are interesting because they necessarily involve recursion. The recursion stops only when we reach a state where the loop condition evaluates to false.

7.6 Symbolic execution of a sketch



Symbolic Evaluation of Commands

Commands have two roles

- Modify the symbolic state
- Generate constraints

$$\mathcal{C}[\![\circ]\!]^\tau : Command \to (\langle \Sigma, \mathcal{P}(\boldsymbol{\Phi})\rangle \to \langle \Sigma, \mathcal{P}(\boldsymbol{\Phi})\rangle)$$

Constraints represent sets of valid ϕ functions

Symbolic Evaluation of Commands

Assignments and Assertion

$$\mathcal{C}[\![x := e]\!]^\tau \langle \sigma, \Phi\rangle = \langle \sigma[x \mapsto \ \mathcal{A}[\![e]\!]^\tau \sigma], \Phi\rangle$$

$$\mathcal{C}[\![\textbf{assert } e]\!]^\tau \langle \sigma, \Phi\rangle = \langle \sigma, \{\phi \in \Phi : \ \mathcal{A}[\![e]\!]^\tau \sigma\phi = 1\}\rangle$$

The basic idea when creating constraints from a sketch is that we want to perform *symbolic execution*. Unlike the standard execution which runs a program and produces states mapping variables to values, our symbolic execution will run a program and produce *symbolic* values and *constraints*.

The formalism is similar from before, but for expressions, the semantics are now a function that takes in a state mapping variable names to symbolic values and producing symbolic values, which are really just symbolic representations of a function from an assignment to holes ϕ to a concrete value. An important thing to note is that the denotation function is parametric on the context τ, which is important in correctly generating constraints for generators.

For example, the figure shows the semantics of a few basic expressions. The most interesting is the semantics for a hole with a unique label $??_i$. Just like we saw before, given an assignment ϕ, the value of the hole is simply whatever ϕ assigns to that hole under the current context.

More interesting still are the semantics of commands. Unlike before, two things happen when we evaluate a command. First, the state may change, as variables are assigned new values. But also, the set of valid assignments may be restricted, for example, by an assertion. So the semantics of a command take in

a state and a representation of a set of valid assignments and produces a new state and a new set of valid assignments. For example, after an assignment statement, the set of valid assignments remains unchanged, but the state is updated with the assigned variable now mapping to a new symbolic value. When an assert is executed, on the other hand, the state remains unchanged, but the set of valid assignments is now restricted to only those assignments that cause the expression to evaluate to true under the current state.

Symbolic Evaluation of Commands

If statement

$$\mathcal{C}[\![\textbf{if } e \textbf{ then } c_1 \textbf{ else } c_2]\!]^\tau \langle \sigma , \Phi \rangle = \langle \sigma' , \Phi' \rangle$$

$$\Phi_t = \{\phi \in \Phi : \ \mathcal{A}[\![e]\!]^\tau \sigma\phi = true\}$$
$$\Phi_f = \{\phi \in \Phi : \ \mathcal{A}[\![e]\!]^\tau \sigma\phi = false\}$$
$$\langle \sigma_1 , \Phi_1 \rangle = \ \mathcal{C}[\![c_1]\!]^\tau \langle \sigma , \Phi_t \rangle$$
$$\langle \sigma_2 , \Phi_2 \rangle = \ \mathcal{C}[\![c_2]\!]^\tau \langle \sigma , \Phi_f \rangle$$
$$\Phi' = (\Phi_1) \cup (\Phi_2)$$
$$\sigma' = \lambda x.\lambda\phi.\ \mathcal{A}[\![e]\!]^\tau \sigma\phi \ ?\ \sigma_1 x\phi \ :\ \sigma_2 x\phi$$

Conditionals



if e

then C1 **else** C2

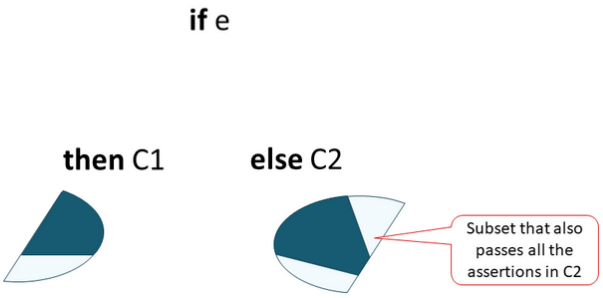
Conditionals



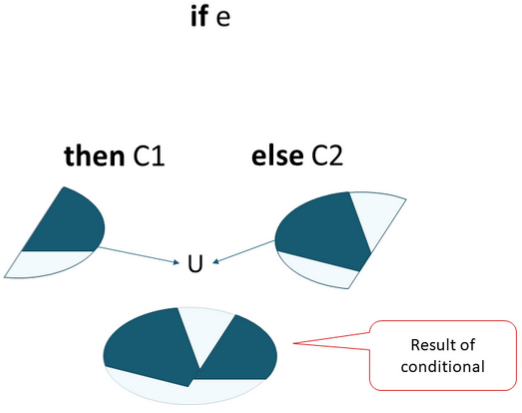
if e

then C1 **else** C2

Conditionals



Conditionals



Symbolic Evaluation of Commands

While loops

$$
\begin{aligned}
W(\langle \sigma, \Phi \rangle) = \; & \mathcal{C}[\![\textbf{while } e \textbf{ do } c]\!]^{\tau} \langle \sigma, \Phi \rangle = \langle \sigma', \Phi' \rangle \\
\Phi_t = \{\phi \in \Phi : \; & \mathcal{A}[\![e]\!]^{\tau} \sigma \phi = true\} \\
\Phi_f = \{\phi \in \Phi : \; & \mathcal{A}[\![e]\!]^{\tau} \sigma \phi = false\} \\
\langle \sigma_1, \Phi_1 \rangle = & \; W(\, \mathcal{C}[\![c]\!]^{\tau} \langle \sigma, \Phi_t \rangle) \\
\Phi' = & \; (\Phi_1) \cup (\Phi_f) \\
\sigma' = & \; \lambda x. \lambda \phi. \; \mathcal{A}[\![e]\!]^{\tau} \sigma \phi \; ? \; \sigma_1 x \phi \; : \; \sigma x \phi
\end{aligned}
$$

The semantics of branches and loops are a little more involved. In the case

of branches, each branch is evaluated on the set of values that satisfy the branch, and the results of the two branches are combined at the end as illustrated by the animation. The state is also modified by the branch in the same way as it was in the case of the simple imperative language.

Loops follow the same logic, but with the caveat that loop evaluation is recursive. This is problematic because unlike the standard execution where we could stop the recursion as soon as we got a state where the condition evaluated to true, in this case we are doing symbolic execution, so even if we wanted to we would not be able to tell when the expression evaluates to true. Note that the definition does not even guard the recursion by a conditional. In principle, we could compute the expression to the right recursively until we reach a fixpoint, that is, sooner or later, additional recursive calls to W will stop contributing anything to the resulting set, and at that point we can stop recursing. In practice, Sketch simply continues this process until we reach a hard-coded limit determined by a compile-time flag "`-bnd-unroll-amnt`".

References

[1] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008. URL http://people.csail.mit.edu/asolar/papers/thesis.pdf.

[2] Emina Torlak and Rastislav Bodík. A lightweight symbolic virtual machine for solver-aided host languages. In Michael F. P. O'Boyle and Keshav Pingali, editors, *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14, Edinburgh, United Kingdom - June 09 - 11, 2014*, pages 530–541. ACM, 2014. doi: 10.1145/2594291.2594340. URL https://doi.org/10.1145/2594291.2594340.

[3] Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength smt solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)*, volume 13243 of *Lecture Notes in Computer Science*, pages 415–442. Springer, 2022. doi: 10.1007/978-3-030-99524-9_24. URL https://doi.org/10.1007/978-3-030-99524-9_24.

[4] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32Nd ACM/IEEE International Conference on Software Engineering - Volume 1*, ICSE '10, pages 215–224, New York, NY, USA, 2010. ACM. ISBN 978-1-60558-719-6. doi: 10.1145/1806799.1806833. URL http://doi.acm.org/10.1145/1806799.1806833.

[5] Armando Solar-Lezama. Program sketching. *STTT*, 15(5-6):475–495, 2013. doi: 10.1007/s10009-012-0249-7. URL https://doi.org/10.1007/s10009-012-0249-7.